FREE PROMPT PACK · CLAUDE CODE · LOCAL AI

Run Jarvis *Fully* Local

Three installs and eight copy-paste prompts. Part One puts three free engines on your own computer: Ollama for the brain, VoiceStudio for the voice, Whisper for the ears. Part Two builds Jarvis on top of them with Claude Code: a 3D galaxy of your notes, a butler who answers from them, a cloned voice, hands-free listening, a memory that grows when you talk, a brain you swap out loud, and a preflight that tells you the truth. No API keys, no subscriptions, nothing leaves the machine. Turn off the Wi-Fi and keep talking.

ZUBAIR TRABZADA

AI WORKSHOP

OCTOBER 2026

How to use this pack

What you need: a Mac with Apple Silicon and 16 GB of memory or more (the voice engine in this pack is Apple-only; the brain and the ears also run on Windows and Linux), Claude Code to do the building (pick Claude Opus 5.5 with /model), Google Chrome, Python 3, and a folder of markdown notes. Everything Jarvis runs on afterwards is free and open source.

Set up in one line. Open a terminal, make an empty folder, and start Claude Code inside it: `mkdir my-jarvis && cd my-jarvis && claude`. Do the three installs in Part One first, then paste Prompt 01.

1 **Claude Code builds it. Your computer runs it.**
The building happens in the cloud, once. What comes out is a folder of plain Python and HTML that runs on your machine with the three local engines. After the build, Jarvis never needs a key.

2 **Install first, prompt second.**
Prompts 01 to 03 expect Ollama, VoiceStudio and Whisper to already answer on your machine. Each install page ends with one command that proves it. Run it before you move on.

3 **Paste each prompt whole, unedited, the first time.**
Every prompt is a complete brief: what to build, the laws it must respect, and how to prove it works. The verification at the end of each one is where the reliability comes from.

4 **Clone only a voice you may use.**
Your own voice, or one you have permission for. The engine in this pack is licensed for any use, including commercial. Cloning a famous voice for anything public is a legal problem, not a technical one.

5 **Expect honest speed, then tune it.**
A local brain on a laptop is slower than a cloud one; on a Mac Studio it can match the cloud. The pack tells you where the time goes and how to claw it back, and Prompt 07 prints your own machine's numbers so you never guess.

6 **Test quietly.**
Test tabs open with ?mute=1 so a background tab never speaks. Every prompt that touches the voice or the ears keeps that law.

Cloud Jarvis versus local Jarvis, honestly

One conversational turn, from the moment you stop talking to the moment Jarvis starts speaking. The cloud column is the Jarvis from the videos. The local column is what this pack builds. How fast the local column feels depends entirely on the machine it runs on, which the note below explains.

	CLOUD JARVIS	LOCAL JARVIS (THIS PACK)
Hearing	The browser's speech recognition, which sends your audio to Google's servers.	**Whisper on your machine.** Quick enough to feel live.
Thinking	Claude Opus, fast, a few cents a reply.	**A local model through Ollama**, free. Slower on a laptop, cloud-fast on a Mac Studio, and the lean prompt from Prompt 04 helps on both.
Speaking	ElevenLabs, fast, a monthly plan.	**A cloned voice through VoiceStudio.** A beat slower than the cloud on a laptop, free.
Privacy	Audio to Google, text to Anthropic, speech to ElevenLabs.	**Nothing leaves the machine.** Prompt 08 proves it.
Quality	The smartest brain available. Follows every rule, lands every joke.	**A good assistant, not a genius** on a laptop-sized model. A Mac Studio runs a 70B-class brain that closes most of the gap.
Cost	Cents per turn plus the voice plan.	**$0.** Electricity.

Speed depends on the machine. A local brain generates words as fast as the chip can read its own memory. On a normal MacBook Pro with an M4 chip the local Jarvis is noticeably slower than the cloud one. On a Mac Studio with an M5 Ultra it is as fast as the cloud, sometimes faster, and it can run a much bigger brain. Same pack, same prompts; the hardware sets the pace.

What stays in the cloud. Live web research, by definition. And heavier agent work (taking apart a website, building a feature) runs on Claude Code, because a laptop-sized model cannot do it well. Prompt 06 gives Jarvis a spoken switch so you can plug the cloud brain back in for those moments and unplug it afterwards. Everything in this pack works with the plug out.

Part One · the installs

Three free engines, each one a desktop app or a Python package, each one answering on a local port that the prompts in Part Two will talk to. Nothing here needs an account. Do them in order; each takes about ten minutes plus the download.

The brain	Ollama runs open models on your machine and exposes them through the same style of endpoint the cloud brains use, so Jarvis cannot tell the difference.
The voice	VoiceStudio clones a voice from a 10 to 20 second clip and speaks any text through it, using an Apple-native engine that is licensed for any use.
The ears	Whisper turns your speech into text on your own chip. Two sizes: a large one for your commands and a small one that only listens for the wake word.

Two laws hold across all three. Pin everything: the model, the engine, the version. "Latest" is how a working Jarvis breaks on a Tuesday. And prove each install with the one command at the bottom of its page before you touch the next one; Claude Code cannot fix an engine that was never there.

YOUR MEMORY	BRAIN TO PULL	WHAT TO EXPECT
8 GB	gemma2:2b	Small talk and simple questions. Keep the heavy thinking for a cloud brain.
16 GB	**qwen3.5:9b**	The sweet spot. A real assistant, about 6 GB on disk.
24 to 36 GB	gemma4:12b	Noticeably smarter, a little slower, with a longer first load.
64 GB and up	a 30B mixture-of-experts or a 70B-class model	Mac Studio territory. Close to cloud quality, and on an M5 Ultra as fast as the cloud, sometimes faster.

INSTALL 01 · THE BRAIN

Ollama and a local model

Best for: the first install. Ten minutes, one download, and a brain that costs nothing per reply.

1. Download Ollama from **ollama.com/download** (Mac, Windows, Linux), open it once, and let it sit in your menu bar. It serves models on port 11434.
2. Pull the brain for your memory size from the table on the previous page. The pull is a one-time download of a few gigabytes.
3. Talk to it once in the terminal, then prove the door Jarvis will use: the OpenAI-compatible endpoint. No key, "ollama" is a placeholder the endpoint ignores.

```
# 2. the brain (pick ONE from the table; this is the 16 GB sweet spot)
ollama pull qwen3.5:9b

# 3a. prove it talks
ollama run qwen3.5:9b "Say hello in one sentence."

# 3b. prove the door Jarvis will use (the same request shape as a cloud brain)
curl http://localhost:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{"model":"qwen3.5:9b","reasoning_effort":"none",
       "messages":[{"role":"user","content":"Say hello in one sentence."}]}'

# expect a JSON reply with a "content" string that is NOT empty.
# if "content" is empty and finish_reason is "length", the model spent its whole budget
# thinking silently. "reasoning_effort":"none" is the switch that stops that; keep it.
```

Make it yours: pull two brains and keep both. A 2B model for the "good morning" chatter and a 9B or 12B for real questions, and Prompt 06 lets you swap between them out loud. Ollama only keeps one loaded at a time, so the first reply after a swap carries a few seconds of loading.

INSTALL 02 · THE VOICE

VoiceStudio and a cloned voice

Best for: the moment Jarvis stops sounding like a laptop. Apple Silicon only for the fast engine.

1. Install the app with the one-line installer below, then open it. On the first run click **Install local runtime** and wait; it downloads a few gigabytes once. The engine then answers on port 3900 whenever the app is open.

2. In **Model Catalogue**, install **CSM 1B** and select the **MLX audio** engine. CSM is Apache-2.0, so you may use what it makes for anything. If the first-run screen insists on its default voice pack, let it download, then pick CSM anyway; the default pack's weights are licensed for non-commercial use only.

3. Make the voice: **Profiles, New, Clone**, with a 10 to 20 second clip of ONE clean speaker, no music, no room echo. Your own voice is the obvious first clone. Name the profile "Jarvis"; Prompt 02 looks it up by that name.

```
# 1. install (Mac). Windows, in PowerShell:  irm https://voicestudio.sh/install | iex
curl -fsSL https://voicestudio.sh/install | sh

# prove the engine is up (the app must be open)
curl http://127.0.0.1:3900/health

# prove the clone speaks: a WAV lands in your folder. Replace PROFILE_ID with the id
# shown by  curl http://127.0.0.1:3900/profiles
curl http://127.0.0.1:3900/v1/audio/speech -H "Content-Type: application/json" \
  -d '{"model":"mlx-community/csm-1b-8bit","voice":"PROFILE_ID","language":"en",
       "response_format":"wav","input":"Good afternoon. The local voice is live."}' \
  -o local-voice-check.wav
```

Make it yours: the clip decides everything. Record it on the mic you will actually use, in the room you will actually sit in, reading two or three calm sentences. A shorter clean clip often clones better than a longer one with a breath or a chair creak in it.

INSTALL 03 · THE EARS

Whisper, on your own chip

Best for: closing the one gap most "local" assistants quietly leave open.

1. Install the Whisper package for your machine: **mlx-whisper** on Apple Silicon (fastest), **faster-whisper** everywhere else. Both are free, open source, and run with no account.
2. Record a five-second test clip of yourself saying a known sentence (QuickTime or Voice Memos, exported as test.wav into your Jarvis folder).
3. Run the proof below. The first run downloads the model once, about 1.5 GB for the large turbo model. The second run is the real speed.

```
# 1. Apple Silicon
python3 -m pip install mlx-whisper

#    Windows, Linux, Intel Mac
python3 -m pip install faster-whisper

# 3. prove it hears you (Apple Silicon). Expect your sentence printed back.
python3 -c "import mlx_whisper; print(mlx_whisper.transcribe('test.wav',
  path_or_hf_repo='mlx-community/whisper-large-v3-turbo')['text'])"

#    the small model Prompt 03 uses for the wake word, warmed the same way:
python3 -c "import mlx_whisper; print(mlx_whisper.transcribe('test.wav',
  path_or_hf_repo='mlx-community/whisper-small-mlx')['text'])"
```

Make it yours: if the large model takes more than two seconds on your five-second clip, use the small one for everything. A fast ear that misses a word now and then beats a perfect ear that makes every sentence wait.

Part Two · the prompts

Eight prompts for Claude Code, in order. The first three plug the three engines into a Jarvis: the galaxy and the local brain, the cloned voice, the local ears. Then the character, the growing memory, the spoken brain switch, the preflight, and the drill that proves nothing leaves the machine.

The brain	A 3D galaxy of your notes with a brain that answers from them, through Ollama, with the prompt built to be cached.
The voice	Every answer spoken in the cloned voice, chunked so the first words land fast and the rest never pauses mid-sentence.
The ears	Hands-free: "Jarvis" wakes him, Whisper hears the command, interrupt words stop him instantly.
The switch	"Switch to Claude" and "back to your local brain", out loud, with a badge that shows which brain and voice are live.
The proof	A preflight that prints your machine's real latency budget, and a drill that lists every outbound connection during a turn.

Two laws hold across all eight prompts. Never trade fluency for first-word speed: a voice that starts fast and then pauses between words sounds broken, while one that starts a second later and flows sounds alive. And every claim is a receipt: Jarvis never says an action happened unless the code handed him proof.

PROMPT 01 · THE FOUNDATION

The Galaxy and the Local Brain

Best for: the first paste. The whole foundation, with a brain that costs nothing, in one pass.

```
Build me a 3D knowledge galaxy from my markdown notes, with a LOCAL brain I can talk to through Ollama. Do the whole thing in one pass, then start it and verify it before you tell me it's done.

MY NOTES: [PASTE THE FULL PATH TO YOUR NOTES FOLDER, or delete this line and write "I have no notes, so generate 30 realistic sample notes about a small business into ./notes first"]

THE INDEXER: build.py, Python 3, standard library only. Scan every .md file and write viewer/graph-data.js containing const GRAPH = {nodes: [...], links: [...]}. Each node: a label from the filename, its parent folder as a group, roughly a 700-character excerpt. Link two notes when one mentions the other's title or they share [[wikilinks]]. CRITICAL: every node gets a numeric id equal to its index in the nodes array.

THE VIEWER: viewer/index.html, single page, 3d-force-graph from a CDN pinned to an exact version (Prompt 08 brings it local). No npm, no build step. Cinematic: black space, a starfield, nodes glowing by group, slow idle rotation. Clicking a node flies the camera to it, lights its neighbours, and opens a side panel with the excerpt.

THE SERVER: server.py, port 4700, bound to 127.0.0.1, serving ONLY the viewer/ folder. Create config.json in the PROJECT ROOT, never inside viewer/: {"brain": {"base_url": "http://localhost:11434/v1", "model": "qwen3.5:9b", "api_key": "ollama", "extra_body": {"reasoning_effort": "none"}}}. Talk to it with the OpenAI-style chat completions request using urllib from the standard library, no SDK, merging extra_body into every request body. The same code must work unchanged against any cloud brain later, by changing only base_url, model and api_key.

THE BRAIN: add POST /chat. Score every note against my question by keyword overlap, weighting title matches higher, take the top 6, and ask the model. System prompt: answer ONLY from these notes, in two or three sentences, and say so plainly when the notes do not cover it. Return {"answer": "...", "nodes": [indexes used]}. Keep a short server-side history so follow-ups work. A clean input bar at the bottom of the viewer shows the answer.

LOCAL BRAIN RULES, because every token costs seconds here, not cents: the system prompt must be identical on every turn and come FIRST, with the retrieved notes and my question LAST, so the model's cache skips re-reading it. Keep the system prompt under 400 words. Never send a thinking-disabled flag; the extra_body switch is what stops silent thinking. Give the reply at least 600 max tokens and retry once with a bigger budget if the text comes back empty.

FINISH PROPERLY: start the server, confirm the page loads and the graph renders, ask /chat a real question and print the answer AND the seconds it took. Then give me three example questions based on my actual note titles.
```

Make it yours: the seconds printed at the end are your baseline. Write them down. Prompt 04 and Prompt 07 are where you win most of them back.

PROMPT 02 · SPEECH

The Cloned Voice

Best for: the moment it stops being a website and starts being someone.

```
Give it the cloned voice from VoiceStudio, which is open on this machine and answers on http://127.0.0.1:3900.

THE VOICE BAY: voice.py, standard library only, soft-imported by server.py so Jarvis boots fine without it. Add
"voice": {"url": "http://127.0.0.1:3900", "engine": "mlx-audio", "model_id": "mlx-community/csm-1b-8bit", "profile":
"Jarvis", "language": "en"} to config.json. At boot, and once per engine run, POST /engines/select with {"family":
"tts", "backend_id": ENGINE, "model_id": MODEL_ID} so the engine is pinned and never silently swapped. Find the profile
id by name from GET /profiles.

THE ENDPOINT: POST /tts with {"text"} returns audio/wav from POST /v1/audio/speech with {"model": MODEL_ID, "input":
text, "voice": PROFILE_ID, "response_format": "wav", "language": LANG}. Cap one request at 600 characters. Cache every
WAV on disk by a hash of text plus profile, so repeated lines like the greeting are instant. Add the header X-Jarvis-
Voice: local:PROFILE_NAME to every reply.

KEEP IT WARM: every 10 idle minutes, render "Standing by." to the engine and throw the audio away, so the model stays
loaded and the first real line of the day is not a 7-second cold start.

THE PLAYER, and the law: split each answer into sentence-packed chunks of about 170 characters, never breaking inside a
sentence. Request the first chunk, play it, and fetch the next chunk while the first one plays, so playback never gaps.
Do NOT cut the first chunk shorter to start faster: a voice that pauses in the middle of a sentence sounds broken, and
I will take a second of extra wait over that every time.

FALLBACK: if GET /health does not answer within one second, speak with the browser's Web Speech API instead and mark
the reply X-Jarvis-Voice: browser. Never fail silently; never crash.

THE MUTE FLAG: if the page URL contains ?mute=1 nothing in that tab ever speaks. Put the check inside the ONE function
that plays audio, so nothing can route around it.

VOICE ROUTES: keep a list of voice routes, each a pattern and a handler, checked BEFORE a sentence goes to the brain.
Add "use your local voice" and "use the browser voice". Later prompts add their own routes to this list.

VERIFY: play a 10-word line and an 80-word answer, print the seconds to first audio for each, confirm the second chunk
was already fetched when the first finished, and confirm a ?mute=1 tab stays silent.
```

Make it yours: clone two profiles, a calm one and an energetic one, and add a route for each. Same brain, different mood, one sentence to switch.

PROMPT 03 · HEARING

The Local Ears

Best for: hands-free, with no audio leaving the room.

```
Give it ears that run on this machine. No browser speech recognition anywhere in this project: it sends audio to a cloud.

THE HEARING BAY: hear.py, using mlx_whisper on Apple Silicon and faster_whisper elsewhere, soft-imported so Jarvis boots without it. Add "hearing": {"model": "mlx-community/whisper-large-v3-turbo", "wake_model": "mlx-community/whisper-small-mlx"} to config.json. Load both models ONCE at boot, keep them in memory, and warm each with one second of silence so the first real transcription is not a cold start. POST /hear takes a WAV body and returns {"text", "seconds"}.

THE MICROPHONE: in the viewer, capture with getUserMedia and encode 16 kHz mono 16-bit WAV in the browser, no ffmpeg. Measure loudness continuously. An utterance starts when loudness crosses a threshold and ends after SILENCE_MS of quiet, a named constant at the top of the file, starting at 700. Ignore anything under 300 ms. Then apply the FINISH WINDOW: buffer the transcript and wait FINISH_MS (start at 900) for more speech before dispatching, because I pause in the middle of sentences. More speech appends and restarts the timer.

THE WAKE WORD: a hands-free toggle. When it is on, every utterance goes to the SMALL model first. If the text starts with "Jarvis" (also "hey Jarvis"), strip it and send the rest to the LARGE model and on to /chat. If nothing follows the name, say "Yes, sir?" and treat the next utterance as the command. Anything without the name is discarded, never logged, never sent anywhere. A push-to-talk button bypasses the wake word and goes straight to the large model.

INTERRUPTING: while Jarvis is speaking keep listening, but only for "stop" and "wait", which stop playback instantly. Raise the threshold while audio plays so his own voice through the speakers does not trigger it, and tell me plainly if my setup needs headphones.

STATUS: a status line that shows listening, heard (with the text), thinking and speaking, so I always know which of us the machine thinks is talking. ?mute=1 tabs still hear, they just never speak.

VERIFY: a test plan with what to say and what I should see, including one mid-sentence pause to prove the finish window, one sentence without the wake word that must be discarded, and the measured seconds from the end of my speech to the transcript on screen.
```

Make it yours: SILENCE_MS and FINISH_MS are the two most personal numbers in the build. Tune them after a day of real use, never in one sitting, and tune them separately: one is how you pause, the other is how you think.

PROMPT 04 · CHARACTER

The Butler, Lean

Best for: a personality that fits in a small brain and a small prompt.

```
Give it a personality that a 9B model can carry and a local prompt can afford.

Rewrite the /chat system prompt so the assistant is a dry, impeccably polite British butler with a razor wit. Rules:
- Address me as "sir", occasionally, not in every sentence.
- Answer in ONE witty sentence plus the facts. Never recite the note back; it is on my screen.
- When my notes do not cover something, say so plainly and with a little dignity. Never invent a source.
- Never claim an action happened unless the code actually did it and handed you a receipt. "Done" when nothing happened is the one unforgivable sin.
- When asked what he costs to run, the honest answer is nothing: he lives on this machine.

THE BUDGET: the entire persona block must be under 300 words, because on a local brain every word of the prompt is paid for in seconds on every turn. Smaller models follow short, concrete rules and ignore long ones, so prefer five sharp rules over fifteen careful ones. Put the persona in ONE clearly commented block at the top of server.py.

STABLE PREFIX: the persona, the rules and the control tags never change between turns and always come first. Anything that changes (the date, the retrieved notes, my question) comes after them. This is what lets the model reuse its cache instead of re-reading the whole prompt every time.

BOOT GREETING: when the page loads it says one short line, "Hello, sir.", and nothing more. Show the real indexed note count ON SCREEN, read from the graph data, never hardcoded.

CONTROL TAGS: when I ask for an action the brain cannot do by talking, it replies with a short tag in square brackets instead of prose, and the server executes the tag. Keep the tag list in the persona block; later prompts add their own. Test that this small model actually emits the tag reliably: ask five times, count the hits, and if it misses more than once, simplify the instruction until it does not.

VERIFY: print the token count of the system prompt and the seconds per reply BEFORE and AFTER this change, then give me five things to say that show off the personality, including one question it cannot answer from my notes.
```

Make it yours: a small brain is a worse actor than a big one, so give it fewer lines. One perfectly dry sentence per answer is a character; three attempted jokes is a model trying too hard.

PROMPT 05 · GROWTH

Total Recall

Best for: the point where the brain grows without you opening an editor.

```
Let me grow the brain by voice.

- When I say or type anything starting with "remember that", POST it to a new /remember endpoint that writes a real
markdown file into a captures/ folder inside my notes directory, with a sensible title from the first few words and
today's date inside the file.
- Add the new node to the running galaxy LIVE, without a page reload: born at the position of its most related existing
node, a brief glow pulse, then fly the camera to it.
- Link it into the graph the same way build.py would, so it is a real citizen of the galaxy and can be a source in the
very next answer.
- Confirm out loud in one line, in character, in the cloned voice.

ONE DOOR FOR NEW NOTES: put the server side in ONE function (write the file, append the node to graph-data.js, add it
to the live search index, return its index) and the viewer side in ONE function (addNote: birth, link, pulse, fly).
Every future feature that creates a note goes through exactly these two.

MAKE THE ANSWER PROVE ITSELF: /chat already returns the indexes it used. Fly the camera to the top source node and open
its panel while the answer is spoken, so I can see which note it came from. If the answer drew on four or more notes,
light the whole cluster instead of flying to one. Small talk never moves the camera: decide whether the question was
about my notes BEFORE you decide whether to move anything.

Two things to get right, because they bite later:
1. Writing a file is not the same as indexing it. The new note must be searchable by /chat in the next question without
re-running build.py.
2. Never let a capture silently fail. If the write fails for any reason, say so out loud. A second brain that quietly
forgets is worse than no second brain.

Then have me test it: I will say "Jarvis, remember that the finish window should be 900 milliseconds", you show me the
new star being born, and I will immediately ask a question that should retrieve it.
```

Make it yours: add trigger phrases that match how you actually talk: "note to self", "don't let me forget", "log this". The capture only works if the phrase is one you would say without thinking.

PROMPT 06 · THE SWITCH

Swap Brains Out Loud

Best for: keeping the cloud as an option you choose, one sentence at a time.

```
Let me swap brains and voices by voice, and always see which ones are live.

THE SHELF: config.json gets a "brains" block of named entries, each with base_url, model, api_key and optional extra_body. Ship it with "local" (the Ollama entry from Prompt 01) and "claude" ({"base_url": "https://api.anthropic.com", "model": "claude-opus-5-5", "api_key": "PUT-YOUR-KEY-HERE"}). The Anthropic entry uses the Messages API shape; keep the two request builders side by side in one small function. If the key is still the placeholder, "switch to Claude" answers in one dry line that the cloud brain has no key, and stays local. I will type the real key into the file myself; never ask me for it in chat.

THE ROUTES: "switch to your local brain", "switch to Claude", "which brain are you on", "go local" (brain, voice and ears all local in one word), "go cloud". Swaps are runtime only: every restart boots on the brain named in config.json "boot_brain", default "local". Confirm each swap in one short line, spoken by the NEW setup.

THE BADGE: a small line under the status that shows BRAIN and VOICE, for example BRAIN local qwen3.5:9b, VOICE local Jarvis. It is hidden by default and appears for 5 seconds whenever either one actually changes, including when the voice falls back to the browser because the engine died. The same voice on every reply must never re-trigger it.

HONESTY: when a request needs the cloud and the cloud is unplugged (live web research, anything the local brain says it cannot do), Jarvis says so in one line instead of pretending. Add that rule to the persona block.

PRELOAD: when I switch to a brain Ollama has not loaded yet, say "one moment, sir" while it loads instead of leaving silence.

VERIFY: in a ?mute=1 tab, run all the routes and show the badge appearing and disappearing, prove a restart boots back to the configured brain, and prove the placeholder key never leaves the machine (grep the viewer and the network log for it).
```

Make it yours: the shelf is where a Mac Studio shines. Add a 70B-class entry next to the 9B and switch between them on camera; the difference in the answers is the whole argument for the bigger machine.

PROMPT 07 · THE ONE THAT MATTERS

The Preflight, with a Latency Budget

Best for: knowing your machine's real numbers instead of guessing them.

```
Build me a preflight harness. This is the most important prompt in this pack, so take your time.

Write preflight.py that runs every LIVE chain end to end and prints a verdict. Not unit tests, not mocks. Real calls against the running system, because the failures that hurt are the ones where every unit test passes and a real chain is dead.

Check, in order, with a tick or a cross for each:
- The server is up and serving the viewer.
- The graph data loads and the node count is greater than zero.
- Ollama answers on /v1/models and the configured brain is in the list. If not, print the exact pull command.
- /chat returns a well-formed answer to a real question, with a nodes array, and the seconds it took.
- VoiceStudio answers /health, the selected engine and model match config.json, and /tts returns a real WAV with its seconds to first byte.
- THE LOOP: render a known sentence through /tts, send that WAV to /hear, and expect the same words back (allow small differences). This proves the ears and the voice agree with each other.
- /remember writes a real file and the note is retrievable by /chat immediately afterwards (then remove the test note).
- The files the browser is actually SERVED match the files on disk.
- config.json is not reachable from the browser. This one must fail loudly.
- A ?mute=1 tab really is silent: the one play function refuses when the flag is set.
- No cloud key appears anywhere under viewer/.

Finish with two lines. First, a LATENCY BUDGET: hear X s, think Y s, speak Z s, total, measured in this run. Second, the summary: N pass, N fail, N warn. Exit non-zero on any failure. Voice and ears missing are WARN, not FAIL, because Jarvis still works through the browser voice without them.

Then run it and show me the output. If anything fails, fix the cause and run it again until it is clean. Do not describe what it would say. Run it.

From now on, whenever you tell me a change is done, run preflight.py first and paste both lines. "Done" means preflight passed.
```

Make it yours: the latency budget is the number to post in the community thread with your machine's name. That thread becomes the buying guide: laptops at one end, Mac Studios at the other.

PROMPT 08 · THE PROOF

The Wi-Fi-Off Drill

Best for: being able to say "nothing leaves this machine" and mean it.

```
Prove that a conversation with Jarvis never leaves this machine, then make it true with the Wi-Fi off.

BRING THE LIBRARIES HOME: the viewer still loads its 3D libraries from a CDN. Download the exact pinned versions once into viewer/vendor/, rewrite the imports to those local files, and remove every remaining reference to any CDN. Record the versions and their source URLs in viewer/vendor/VERSIONS.txt. After this, the page must render with no network at all.

THE WATCHER: wrap the server's outbound HTTP so every connection it opens is logged with the host and the reason, and read the browser's resource timing entries after a turn. Add GET /drill/connections that returns both lists for the last turn.

THE DRILL, as a script: start a fresh turn, send one real question through /chat, speak the answer through /tts, feed that WAV back through /hear, then fetch /drill/connections and PRINT every host that is not localhost or 127.0.0.1. The drill passes only when that list is empty. If "go cloud" is active, the drill must FAIL and name api.anthropic.com, which proves the watcher works.

THE HONEST LIST: the drill ends by printing the features that still need the internet, read from the code, not from memory: the cloud brain if a key is set, live web research if it exists, and anything else you find. No feature is allowed to need the internet without being on this list.

PREFLIGHT GROWS TWO CHECKS: no CDN reference anywhere under viewer/, and the drill passes in local mode.

VERIFY: run the drill in local mode and paste the empty host list, run it in cloud mode and paste the failure, run preflight and paste both lines. Then give me a four-step script for the on-camera version: turn off Wi-Fi, reload the page, ask one question whose answer lives in one note, and what I should see and hear.
```

Make it yours: the on-camera version is the whole video in fifteen seconds. Wi-Fi icon crossed out in the corner, a question, the galaxy flying to the note, the answer in a cloned voice. Nobody can argue with that shot.

Seven things I learned taking Jarvis local

1 **Benchmark the voice engine before you commit.**
The first local engine I tried ran many times slower than real time, unusable for conversation. The second one, CSM on Apple's MLX, renders faster than real time once it is warm. Same app, same Mac, one setting.

2 **A local brain can think itself to death.**
The 9B model spent its entire token budget reasoning silently and returned an empty answer. The usual off switches were ignored; only reasoning_effort set to none worked. Every Ollama entry in this pack carries it.

3 **The slow part was the prompt, not the Mac.**
A full-size Jarvis sends thousands of tokens of personality and memory on every turn, and a laptop reads that slowly. A 300-word persona with a stable prefix is worth more than a new computer.

4 **Starting fast made it sound broken.**
I tried playing the first sentence as soon as it existed. Jarvis paused mid-sentence while the next chunk rendered, and it sounded like a bad phone line. Sentence-packed chunks that prefetch during playback fixed it. Fluency beats first-word speed.

5 **Hearing was the hidden cloud.**
Brain local, voice local, and the browser's speech recognition was still sending my audio to Google. Most "local" assistants stop one step short. Whisper is the step.

6 **Licences live in the weights.**
The voice app's default model is non-commercial; the CSM model is Apache-2.0. Same app, two very different sets of rights. Pin the engine on every call so a default never sneaks back in.

7 **Memory bandwidth is the spec that matters.**
A local brain generates words as fast as the chip can read its own memory. A MacBook Pro with an M4 chip feels slower; a Mac Studio with an M5 Ultra reads memory several times faster, and on that machine a local Jarvis answers as fast as the cloud one, sometimes faster. That, not the amount of memory, is what the bigger machine buys.

> **The honest line.** Speed depends entirely on the machine. On a normal MacBook Pro the local Jarvis is noticeably slower than the cloud one. On a Mac Studio with an M5 Ultra it is as fast or faster, with a bigger brain, and it still costs nothing per reply and sends nothing anywhere.

Where to go from here

Do the three installs in an evening, run the eight prompts over a weekend, and live with the local butler for a week before you judge him. He is slower and simpler than the cloud one. He is also yours: no bill, no account, no audio in anyone's data centre. The first time you ask a question with the Wi-Fi off and the galaxy flies to the note, you will understand why that matters.

1 **Post your latency budget.**
The free community has a thread for every pack. Your machine, your three numbers from Prompt 07, and the brain you picked. That thread becomes the buying guide for everyone after you.

2 **Try a bigger brain.**
If you have the memory, pull a 12B or a 30B model and put it on the shelf next to the 9B. Ask both the same five questions. The difference tells you what the extra memory is actually buying.

3 **Build the thing a cloud assistant cannot be.**
Private by construction. A Jarvis that reads your contracts, your client notes or your medical folder and never sends a byte anywhere is a product in its own right, for a whole class of businesses that are not allowed to use the cloud ones.

Want the finished Jarvis without building it?

Inside the paid AI Workshop community you get the full Jarvis build I run every day, with a five-minute install and no code, plus the classroom, the live calls, and a team that helps when something breaks.

Free community: skool.com/aiworkshop-lite
Full workshop: skool.com/aiworkshop
YouTube: Zubair Trabzada | AI Workshop